

Quack Quack What? — Architecture Document

Version: 1.0

Date: March 2026

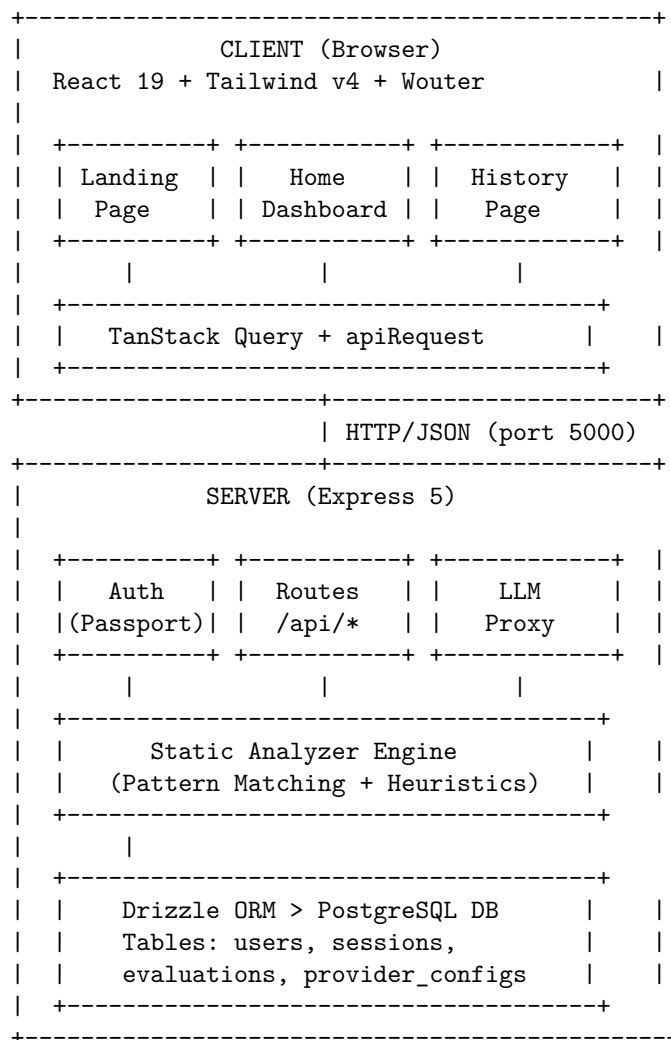
Author: Taashi Manyanga

Trademark: © Taashi Manyanga 2026

1. System Overview

Quack Quack What? is a full-stack web application built on a monorepo architecture with a shared type system between the frontend and backend. The system ingests code from multiple sources, runs static analysis, and provides interactive LLM-powered refinement.

2. High-Level Architecture



3. Component Architecture

3.1 Frontend Components

```
App.tsx (Root)
+-- Router (wouter)
|   +-- Landing Page
|   +-- Home Page
|       +-- SkillImporter (GitHub/URL/Local/Paste tabs)
|       +-- AnalysisSim (animated scan progress)
|       +-- EvaluationReport (scores + findings + download)
|           +-- SkillChat (LLM-powered refinement)
|           +-- SettingsModal (AI provider config)
|   +-- History Page (auth-gated)
|   +-- Readme Page (documentation)
+-- Toaster (notifications)
+-- Trademark Badge (fixed position)
```

3.2 Backend Modules

```
server/
+-- index.ts          > Express app bootstrap, middleware, Vite setup
+-- db.ts             > PostgreSQL connection pool (shared)
+-- routes.ts         > All API route definitions + auth setup
+-- storage.ts        > Drizzle ORM CRUD operations (IStorage interface)
+-- analyzer.ts       > Static code analysis engine
+-- llm.ts            > Multi-provider LLM adapter
+-- static.ts         > Production static file serving
+-- vite.ts           > Development Vite middleware
+-- replit_integrations/
    +-- auth/
        +-- index.ts      > Auth module exports
        +-- replitAuth.ts > OIDC setup, Passport strategies, middleware
        +-- storage.ts    > User CRUD (upsert on login)
        +-- routes.ts     > /api/auth/user endpoint
```

4. Data Flow Diagrams

4.1 Skill Evaluation Flow

See: `diagrams/evaluation-flow.mmd`

1. User submits a skill source (GitHub URL, web URL, file, or paste)
2. Frontend sends POST `/api/fetch-skill` (for GitHub/URL sources)
3. Backend fetches the raw source code
4. Frontend sends POST `/api/evaluate` with the skill code
5. Backend runs the static analyzer (pattern matching + heuristics)
6. Results are stored in the `evaluations` table
7. Response returns scores and findings to the frontend
8. Frontend renders the `EvaluationReport` component

4.2 Authentication Flow

See: `diagrams/auth-flow.mmd`

1. User clicks “Sign In” > redirected to `/api/login`

2. Express initiates OpenID Connect flow with Replit as the identity provider
3. User authenticates via Google, GitHub, Apple, or email/password
4. Replit redirects to `/api/callback` with authorization code
5. Backend exchanges code for tokens, extracts user claims
6. User record upserted in `users` table
7. Session created in `sessions` table (PostgreSQL-backed)
8. User redirected to `/` with active session cookie

4.3 LLM Chat Flow

See: `diagrams/llm-chat-flow.mmd`

1. User enters a use-case description in the SkillChat component
2. Frontend sends POST `/api/chat` with message, skill code, evaluation summary
3. Backend retrieves the configured AI provider from `provider_configs`
4. Backend constructs a system prompt bundling code + analysis + user message
5. Request is routed to the appropriate LLM API (OpenAI, Anthropic, etc.)
6. LLM returns structured JSON: `{fit, feedback, report, refinedCode}`
7. Response rendered in the chat interface with code highlighting

5. Authentication Architecture

5.1 Identity Providers

- **Protocol:** OpenID Connect (OIDC)
- **Provider:** Replit Auth (wraps Google, GitHub, Apple, email/password)
- **Library:** Passport.js with `openid-client` strategy

5.2 Session Management

- **Store:** PostgreSQL via `connect-pg-simple`
- **TTL:** 7 days
- **Cookie:** HttpOnly, Secure, server-side session ID only
- **Refresh:** Automatic token refresh via `refresh_token` grant

5.3 Authorization Model

Route	Auth Required	Scope
POST <code>/api/evaluate</code>	No (Guest OK)	userId set if authenticated
POST <code>/api/fetch-skill</code>	No	Public
POST <code>/api/chat</code>	No	Public
GET <code>/api/history</code>	Yes	Own evaluations only
GET <code>/api/evaluations</code>	Yes	Own evaluations only
GET <code>/api/evaluations/:id</code>	Partial	Own or guest evaluations only
GET/POST <code>/api/provider-config</code>	No	Global config

6. Static Analysis Engine

6.1 Pattern Categories

Category	Pattern Count	Examples
Dangerous Patterns	8+	<code>eval()</code> , <code>exec()</code> , <code>Function()</code> , <code>__proto__</code> , <code>process.env</code> access
Secret Patterns	6+	API keys (<code>sk-</code> , <code>AKIA</code>), tokens, passwords, connection strings
Quality Patterns	6+	Empty catch blocks, <code>console.log</code> , missing types, <code>TODO/FIXME</code>

6.2 Scoring Algorithm

- Each finding has a severity weight (5-25 points)
- Scores start at 100 and are penalized per finding
- Minimum score is 0
- Heuristic bonuses: error handling (+5), TypeScript types (+5), comments density (+5)

6.3 Overall Score Calculation

$\text{overallScore} = (\text{safetyScore} \times 0.4) + (\text{securityScore} \times 0.3) + (\text{qualityScore} \times 0.3)$

7. LLM Provider Architecture

7.1 Supported Providers

Provider	Auth Method	Model Examples
OpenAI	API Key (sk-...)	gpt-4o, gpt-4-turbo
Anthropic	API Key (sk-ant-...)	claude-3-opus, claude-3-sonnet
AI Foundry	API Key + Target URI	Custom endpoints
GCP Vertex AI	Bearer Token	gemini-pro, PaLM 2
AWS Bedrock	AWS Access Keys + Region	Claude, Titan

7.2 Request Flow

```
Client > POST /api/chat > routes.ts > callLLM(config, messages) > Provider API
                                                                    ↓
Client ← JSON response ← routes.ts ← parsed JSON ←----- LLM Response
```

8. Database Architecture

8.1 Entity Relationship

See: `diagrams/database-erd.mmd`

8.2 Tables

Table	Purpose	Key Fields
<code>users</code>	Authenticated user profiles	<code>id</code> , <code>email</code> , <code>firstName</code> , <code>lastName</code> , <code>profileImageUrl</code>
<code>sessions</code>	Server-side session storage	<code>sid</code> , <code>sess (JSONB)</code> , <code>expire</code>
<code>evaluations</code>	Evaluation results	<code>id</code> , <code>userId</code> , <code>scores</code> , <code>findings (JSON)</code> , <code>skillCode</code>
<code>provider_configs</code>	LLM provider credentials	<code>id</code> , <code>provider</code> , <code>apiKey</code> , <code>model</code>

8.3 Connection Management

- Single `pg.Pool` instance created in `server/db.ts`
- Shared across application storage and auth storage
- Drizzle ORM wraps the pool for type-safe queries

9. Security Architecture

9.1 Transport Security

- HTTPS enforced (secure cookies)
- Trust proxy enabled for Replit's reverse proxy

9.2 Session Security

- `HttpOnly` cookies (no JS access)
- Server-side session storage (no sensitive data in cookies)
- Session expiry with automatic token refresh
- OIDC token validation on protected routes

9.3 Data Access Control

- User evaluations scoped by `userId`
- Guest evaluations stored with `userId = null`
- Ownership checks on individual evaluation access
- Auth middleware (`isAuthenticated`) for protected endpoints

9.4 Input Validation

- Zod schemas validate all API request bodies
- Drizzle parameterized queries prevent SQL injection
- Source URL validation before fetching

10. Deployment Architecture

- **Platform:** Replit
- **Server:** Single Express process serving API + static frontend
- **Port:** 5000 (Replit's exposed port)
- **Database:** Replit-managed PostgreSQL
- **Build:** Vite (frontend) + esbuild (backend) for production
- **Dev:** `tsx` for hot-reloading, Vite dev server middleware